

Onyx Messenger

Application de messagerie instantanée minimaliste, moderne et performante développée avec Next.js et Express.js.

Aperçu du projet

Onyx Messenger est une application de chat en temps réel qui permet aux utilisateurs de s'inscrire, se connecter et échanger des messages instantanément. L'application affiche également le statut en ligne des utilisateurs et l'historique des conversations.

Fonctionnalités principales

- Authentification (inscription et connexion)
- Liste des utilisateurs avec statut en ligne/hors ligne
- Messagerie instantanée en temps réel
- Interface utilisateur moderne et minimaliste
- Communication bidirectionnelle via WebSockets

Architecture

Le projet est divisé en deux parties principales :

Backend (Express.js)

- **Clean Architecture** : Organisation en couches distinctes (Domain, Application, Infrastructure, Presentation)
- **TypeScript** : Typage fort pour une meilleure maintenabilité
- **Express.js** : Framework web pour Node.js
- **Prisma ORM** : Gestion de la base de données
- **WebSockets** : Communication en temps réel
- **JWT** : Authentification sécurisée

Frontend (Next.js)

- **App Router** : Utilisation du nouveau système de routage de Next.js
- **TypeScript** : Typage fort pour une meilleure maintenabilité
- **Redux Toolkit** : Gestion de l'état global
- **RTK Query** : Gestion des requêtes API et du cache

- **WebSockets** : Communication en temps réel
- **Tailwind CSS** : Styling moderne et responsive

Installation

Prérequis

- Node.js (version LTS recommandée)
- npm ou yarn
- PostgreSQL (pour la production)

Configuration du Backend

1. Accédez au dossier backend : `bash cd onyx-messenger/backend`
2. Installez les dépendances : `bash npm install`
3. Créez un fichier `.env` basé sur `.env.example` : `bash cp .env.example .env`
4. Modifiez le fichier `.env` avec vos propres valeurs : `PORT=3001 JWT_SECRET=votre-cle-secrete-jwt DATABASE_URL=postgresql://postgres:postgres@localhost:5432/onyx_messenger NODE_ENV=development`
5. Pour le développement, vous pouvez utiliser SQLite au lieu de PostgreSQL en modifiant le fichier `prisma/schema.prisma` : `prisma datasource db { provider = "sqlite" url = "file:./dev.db" }`
6. Générez le client Prisma et exécutez les migrations :
`bash npx prisma generate npx prisma migrate dev --name init`
7. Démarrez le serveur de développement : `bash npm run dev`

Configuration du Frontend

1. Accédez au dossier frontend : `bash cd onyx-messenger/frontend`
2. Installez les dépendances : `bash npm install`
3. Créez un fichier `.env.local` : `bash touch .env.local`
4. Ajoutez l'URL de l'API backend : `NEXT_PUBLIC_API_URL=http://localhost:3001`
5. Démarrez le serveur de développement : `bash npm run dev`

6. Accédez à l'application dans votre navigateur à l'adresse `http://localhost:3000`

Déploiement en production

Backend

1. Assurez-vous d'avoir un serveur PostgreSQL configuré et accessible
2. Mettez à jour le fichier `.env` avec les informations de connexion à la base de données
3. Générez le client Prisma et exécutez les migrations :
`bash npx prisma generate npx prisma migrate deploy`
4. Construisez l'application : `bash npm run build`
5. Démarrez le serveur : `bash npm start`

Frontend

1. Mettez à jour le fichier `.env.local` avec l'URL de l'API backend en production
2. Construisez l'application : `bash npm run build`
3. Démarrez le serveur : `bash npm start`

Structure du projet

Backend

```
backend/
├── prisma/           # Schéma et migrations Prisma
├── src/
│   ├── domain/       # Entités et règles métier
│   │   ├── entities/  # Modèles de données purs
│   │   ├── application/ # Logique applicative
│   │   │   ├── interfaces/ # Interfaces pour l'inversion de dépendances
│   │   │   ├── use-cases/  # Cas d'utilisation
│   │   ├── infrastructure/ # Implémentations concrètes
│   │   ├── database/      # Repositories Prisma
│   │   ├── auth/         # Services d'authentification
│   │   ├── websocket/    # Service WebSocket
│   │   └── presentation/  # Point d'entrée de l'application
│   └── api/           # Routes et contrôleurs Express
├── config/           # Configuration de l'application
└── index.ts          # Point d'entrée principal
```

Frontend

```
frontend/
├── public/           # Fichiers statiques
├── src/
│   ├── app/         # Routes Next.js (App Router)
│   ├── components/  # Composants React
│   │   ├── ui/      # Composants UI génériques
│   │   ├── auth/    # Composants liés à l'authentification
│   │   ├── chat/    # Composants liés au chat
│   │   └── layout/   # Composants de mise en page
│   ├── hooks/       # Hooks personnalisés
│   ├── lib/         # Bibliothèques et utilitaires
│   ├── store/       # Configuration Redux
│   │   ├── slices/  # Slices Redux
│   │   └── api/     # Configuration RTK Query
│   ├── styles/      # Styles globaux
│   └── utils/       # Fonctions utilitaires
└── next.config.js   # Configuration Next.js
```

Choix techniques

Backend

- **Clean Architecture** : Permet une séparation claire des responsabilités et facilite la maintenance et les tests
- **Prisma** : ORM moderne qui offre une excellente expérience développeur et une forte typographie
- **WebSockets** : Permet une communication bidirectionnelle en temps réel
- **JWT** : Standard pour l'authentification qui fonctionne bien avec les architectures sans état

Frontend

- **Next.js avec App Router** : Framework React moderne avec un système de routage avancé
- **Redux Toolkit** : Simplifie la gestion de l'état global avec moins de code boilerplate
- **RTK Query** : Gestion efficace du cache et des requêtes API
- **Tailwind CSS** : Framework CSS utilitaire qui permet un développement rapide et cohérent

Améliorations futures

- Ajout de fonctionnalités de groupe de discussion
- Support pour les médias (images, fichiers)
- Indicateurs de frappe
- Notifications push
- Mode hors ligne avec synchronisation
- Tests unitaires et d'intégration
- Déploiement avec Docker

Licence

Ce projet est sous licence MIT.